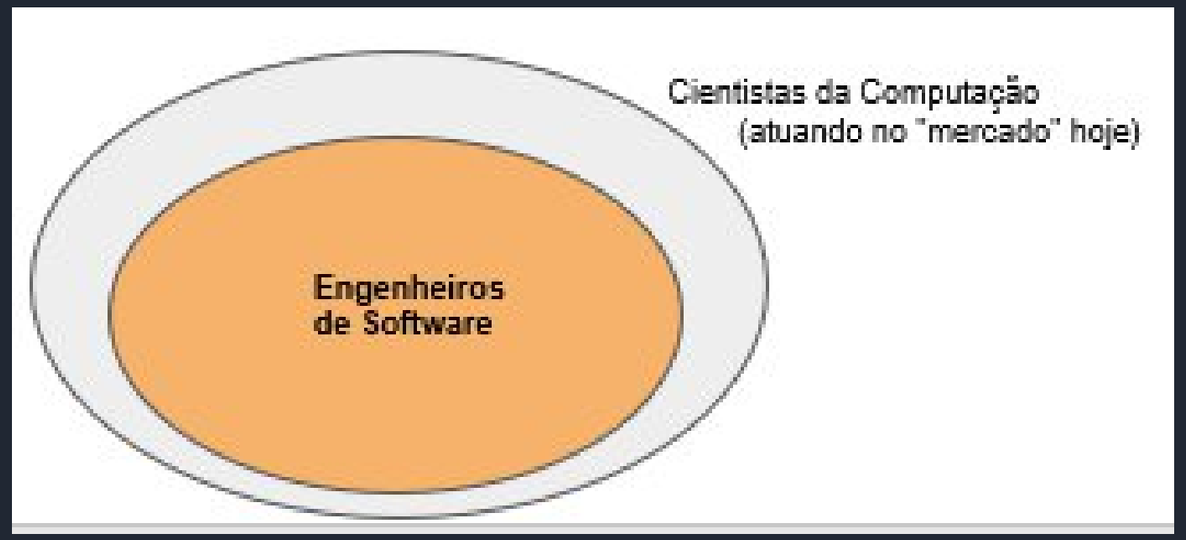
Engenharia de Software Moderna

Licença CC-BY; permite copiar, distribuir, adaptar etc; porém, **créditos devem ser dados ao autor dos slides Prof. Marco Tulio Valente**

AL

Esse conjunto de slides foi adaptado e criado novos slides por Aimar Lopes

Importância da Modelagem de Software E Arquitetura de sistemas



Cientistas da Computação

Precisam conhecer de engenharia de software.

Engenheiros de Software

Existe grande chance de você ser Engenheiro de Software (ou full stack developer, frontend developer, arquiteto, etc)

"Certos sistemas estão colocando demandas que estão além das nossas capacidades... Em algumas aplicações não existe uma crise... Mas **estamos tendo dificuldades com grandes aplicações."**

Conferência da OTAN (Alemanha, 1968)

Esta foi a primeira vez que o termo "Engenharia de Software" foi usado. Aconteceu durante a **Working Conference on Software Engineering** realizada na Alemanha em 1968, organizada pela OTAN.

Definição de Engenharia de Software

A Engenharia de Software é uma área da Computação destinada a investigar os desafios e propor soluções que permitam desenvolver sistemas de software — principalmente aqueles mais complexos e de maior tamanho — de forma produtiva e com qualidade.

O que se estuda em ES?

1 **Engenharia de Requisitos**

Estudar como coletar, analisar, especificar e gerenciar os requisitos de software.

2 **Projeto de Software**

Aprender técnicas de design e arquitetura para criar soluções de software eficientes e escaláveis.

3 **Construção de Software**

Desenvolver habilidades de codificação, integração e implantação de sistemas de software.

4 **Testes de Software**

Estudar métodos e ferramentas para garantir a qualidade e a confiabilidade do software.

5 **Manutenção de Software**

Aprender técnicas para modificar, atualizar e corrigir sistemas de software ao longo do tempo.

6 **Gerência de Configuração**

Entender como controlar e gerenciar as mudanças no código-fonte e na documentação do software.

7 **Gerência de Projetos**

Desenvolver habilidades de planejamento, organização e controle de projetos de software.

O que se estuda em ES? (cont.)

Processos de Software

O estudo dos processos de software envolve a compreensão das etapas e atividades necessárias para o desenvolvimento de sistemas de software de alta qualidade. Isso inclui tópicos como ciclo de vida do software, modelos de processo, gerenciamento de projetos e melhoria de processos.

Modelos de Software

Os modelos de software são representações abstratas de sistemas de software que ajudam os engenheiros a entender, analisar e projetar soluções. Isso inclui tópicos como modelagem de requisitos, design de arquitetura, modelagem de dados e modelagem de comportamento.

Qualidade de Software

A qualidade de software é um aspecto fundamental da Engenharia de Software. Isso envolve o estudo de técnicas de garantia de qualidade, testes, verificação e validação, além de métricas e modelos de qualidade.

Prática Profissional

A prática profissional em Engenharia de Software abrange tópicos como ética, gerenciamento de equipes, comunicação, negociação e liderança. Esses aspectos são essenciais para o sucesso de projetos de software em ambientes corporativos.

E também cuidado: Não Existe Bala de Prata

Frederick Brooks, em seu artigo "No Silver Bullet - Essence and Accidents of Software Engineering", publicado na IEEE Computer em 1987, argumenta que não existe uma solução mágica ou uma "bala de prata" que resolverá todos os problemas da Engenharia de Software.

Ele enfatiza que o desenvolvimento de software é uma tarefa inerentemente complexa e que não há uma abordagem única que possa resolver todos os desafios enfrentados pelos profissionais da área.

Frederick Brooks. No Silver Bullet - Essence and Accidents of Software Engineering. IEEE Computer, 1987. Imagem de: https://twitter.com/zejko_obren/status/909014665802574336

Processo de Software

A alma do Sistema (app)

- ✓ Um conjunto estruturado de atividades necessárias para desenvolver um sistema de software.
- ✓ Existem vários processos de desenvolvimento de software diferentes mas todos envolvem:

E ➡ D ➡ V ➡ E

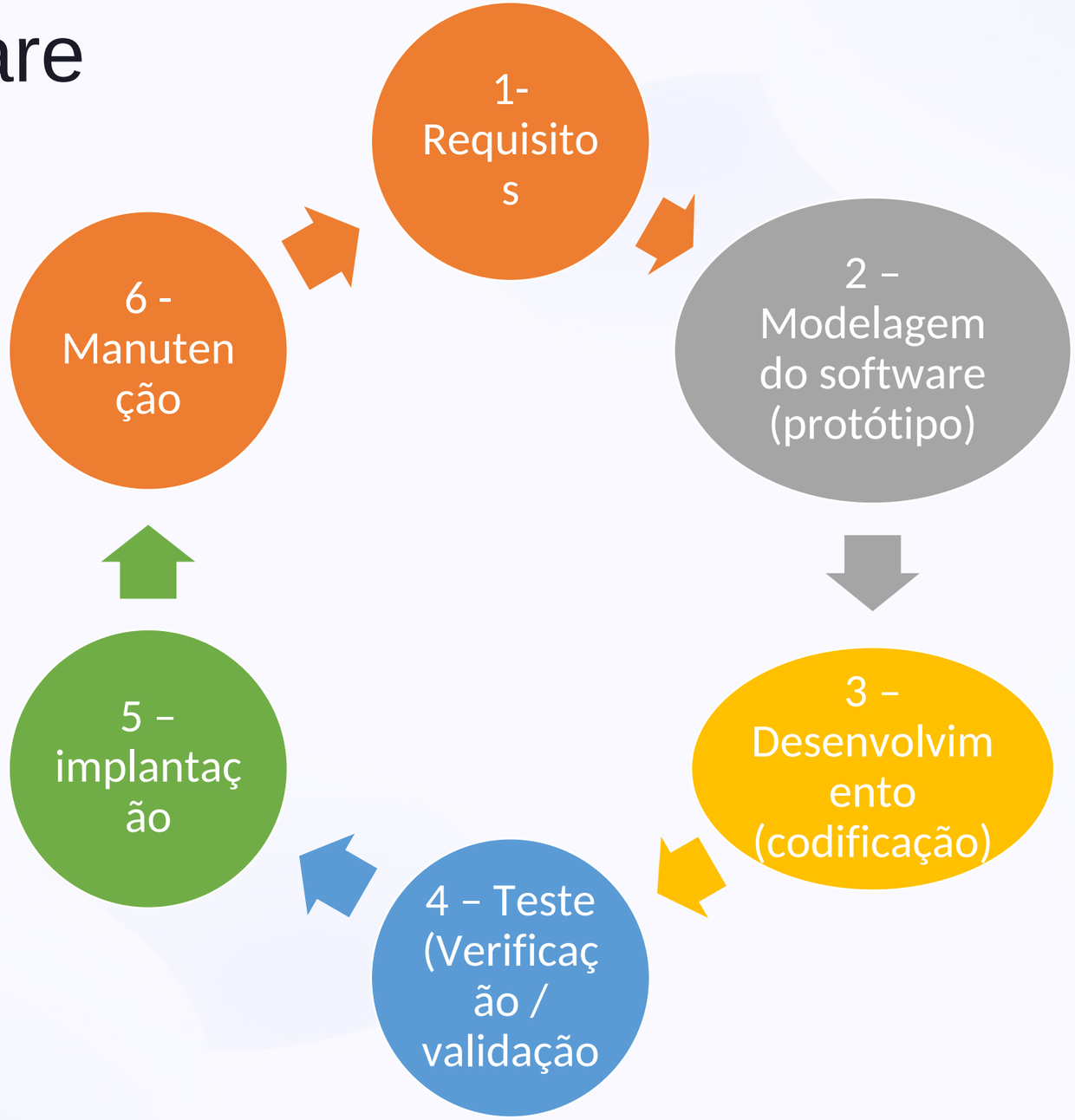
AL

- ✓ **Especificação** – definição do quê o sistema deve fazer (requisitos);
- ✓ **Desenvolvimento** - Projeto e Implementação (codifica) – definição da organização do sistema e implementação do sistema;
- ✓ **Validação** – checagem de que o sistema faz o que o cliente deseja;
- ✓ **Evolução** – evolução em resposta a mudanças nas necessidades do cliente.

Um modelo de processo de desenvolvimento de software é uma representação abstrata de um processo.

Ele apresenta uma descrição do processo de uma perspectiva em particular.

Processos de Desenvolvimento de Software



Made with Gamma

Processos de Desenvolvimento de Software

detalhado

1. Ler o documento de PI
2. Entender o desafio (problema/contexto)
3. Levantar o requisitos (Funcionais e não funcionais)
4. Design (projeto) / protótipo
5. Modelagem do software
6. Protótipo
7. Desenvolvimento (codificação – linguagem)
8. Teste
9. Verificação (confronta com os requisitos)
10. Validação (teste do cçiente / usuário)
11. Implantação (DevOps)
12. Manutenção (atualização. Correção(bug), Refactoring, preventiva)

Made with Gamma

Requisitos de Software

Definição de Requisitos

Requisitos: o que sistema deve fazer para atender aos seus clientes com qualidade de serviço

Engenharia de Requisitos

Engenharia de Requisitos: atividades onde os requisitos de um sistema são especificados, analisados, documentados e validados

Requisitos Funcionais vs Não-Funcionais

Requisitos Funcionais

"o que" um sistema deve fazer - Funcionalidades ou serviços ele deve implementar.

Requisitos Não-Funcionais

"como" um sistema deve operar - Sob quais "restrições" e qualidade de serviço.

Exemplos de Requisitos Não-Funcionais

Alguns exemplos de requisitos não-funcionais importantes a serem considerados no desenvolvimento de um sistema incluem:

AL **by Aimar Martins Lopes**

Exemplos de Requisitos Não-Funcionais

Desempenho

dar o saldo da conta em 2 segundos

Disponibilidade

estar no ar 99.99% do tempo

Capacidade

armazenar dados de 1M de clientes

Tolerância a falhas

continuar operando se São Paulo cair

Segurança

criptografar dados trocados com as agências

Projeto de Software

Definição dos principais componentes

de um sistema e de suas interfaces.

Propriedades de projeto

Princípios de projetos

Padrões de projeto

Vamos estudar os padrões de projeto.

Construção de Software

Padrões de nome

A adoção de padrões de nomenclatura claros e consistentes é essencial para melhorar a legibilidade e manutenibilidade do código.

Implementação (codificação) do sistema

A implementação, ou codificação, é a etapa crucial do desenvolvimento de software onde o sistema é efetivamente construído a partir dos requisitos e do projeto definidos anteriormente.

Ferramentas: IDEs, depuradores, etc

Durante a construção, os desenvolvedores utilizam uma variedade de ferramentas, como IDEs (Integrated Development Environments) e depuradores, para facilitar o processo de codificação e garantir a qualidade do software.

Algoritmos e estruturas de dados

A escolha e implementação adequada de algoritmos e estruturas de dados são fundamentais para garantir a eficiência e o desempenho do sistema desenvolvido.

Bibliotecas e frameworks

Os desenvolvedores também se apoiam em bibliotecas e frameworks pré-existentes para acelerar o desenvolvimento e evitar a reinvenção da roda.

Testes de Software

1

Verificação de Resultados

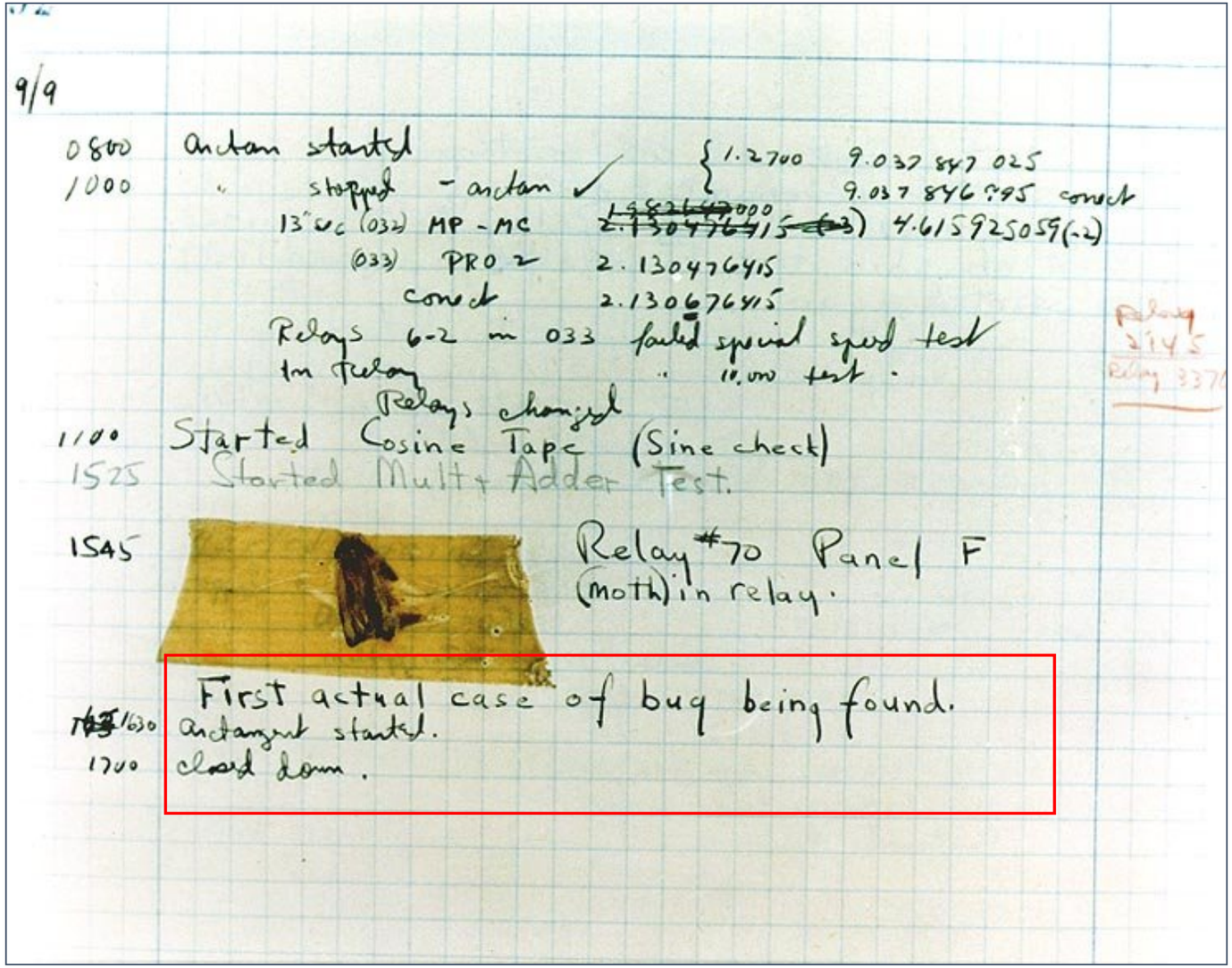
Testes de software verificam se um programa apresenta um resultado esperado, ao ser executado com casos de teste.

2

Tipos de Testes

Testes de software podem ser manuais ou automatizados, sendo o foco neste contexto os testes automatizados.

Um dos primeiros bugs (diário de Grace Hopper, 1947)



Verificação vs Validação

Verificação

Estamos implementando o sistema corretamente? De acordo com os requisitos e especificações.

Validação

Estamos implementando o sistema correto? O sistema que os clientes querem. Testes de aceitação com os usuários.

Manutenção de Software

Tipos de Manutenção

1. Corretiva
2. Preventiva
3. Adaptativa
4. Refactoring
5. Evolutiva

Descrição

A manutenção de software é essencial para garantir que o sistema continue funcionando de forma eficiente e atenda às necessidades dos usuários. Existem diferentes tipos de manutenção, cada um com seus próprios objetivos e abordagens:

- **Manutenção Corretiva:** Visa corrigir erros e defeitos identificados no software.
- **Manutenção Preventiva:** Tem como objetivo evitar a ocorrência de problemas, realizando melhorias e atualizações regulares.
- **Manutenção Adaptativa:** Envolve a adaptação do software a novos requisitos ou ambientes operacionais.
- **Refactoring:** Consiste em melhorar a estrutura interna do software, sem alterar seu comportamento externo.
- **Manutenção Evolutiva:** Visa adicionar novas funcionalidades ou melhorar as existentes, a fim de atender às necessidades em constante evolução dos usuários.

Refactoring

Objetivo

A refatoração é uma manutenção para incrementar a manutenibilidade do software, ou seja, não corrige bugs nem implementa novas funcionalidades. Ela visa melhorar a estrutura interna do código sem alterar seu comportamento externo.

Exemplos

- Renomear variável, classe, etc.
- Extrair função
- Mover função

Gerência de Configuração

1

Controle de Versões

Todo software é desenvolvido usando um sistema de controle de versões (exemplo: git), que atua como uma "fonte da verdade" sobre o código e permite recuperar versões antigas.

2

Apêndice sobre Git

Veja o apêndice sobre git para mais informações sobre esse sistema de controle de versões.

Gerência de Projetos

Negociação de Contratos

Atividades como definição de prazos, valores e cronograma são essenciais na negociação de contratos para projetos de software.

Gerência de Recursos Humanos

A contratação e treinamento da equipe de desenvolvimento fazem parte das responsabilidades da gerência de projetos.

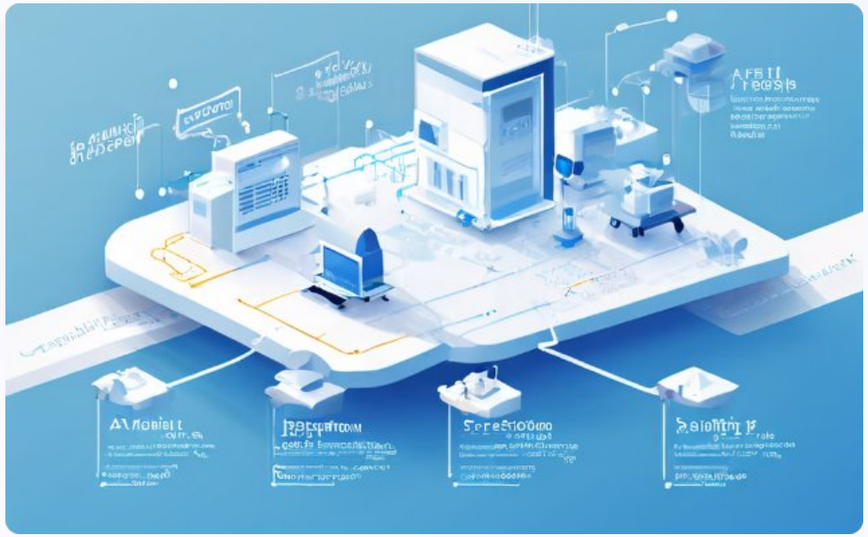
Gerenciamento de Riscos

Identificar e mitigar riscos durante o projeto é uma atividade crucial para a gerência de projetos.

Acompanhamento de Mercado

Acompanhar a concorrência, ações de marketing e outros fatores externos é importante para o sucesso do projeto.

Processos de Desenvolvimento de



Modelo Waterfall (cascata)

Um processo de software define quais atividades devem ser seguidas para construir um sistema de software. O modelo Waterfall, também conhecido como "cascata", é um dos principais modelos de processo de software.

Modelo Ágil

Além do modelo Waterfall, o modelo Ágil, também chamado de incremental ou iterativo, é outro dos principais modelos de processo de software.

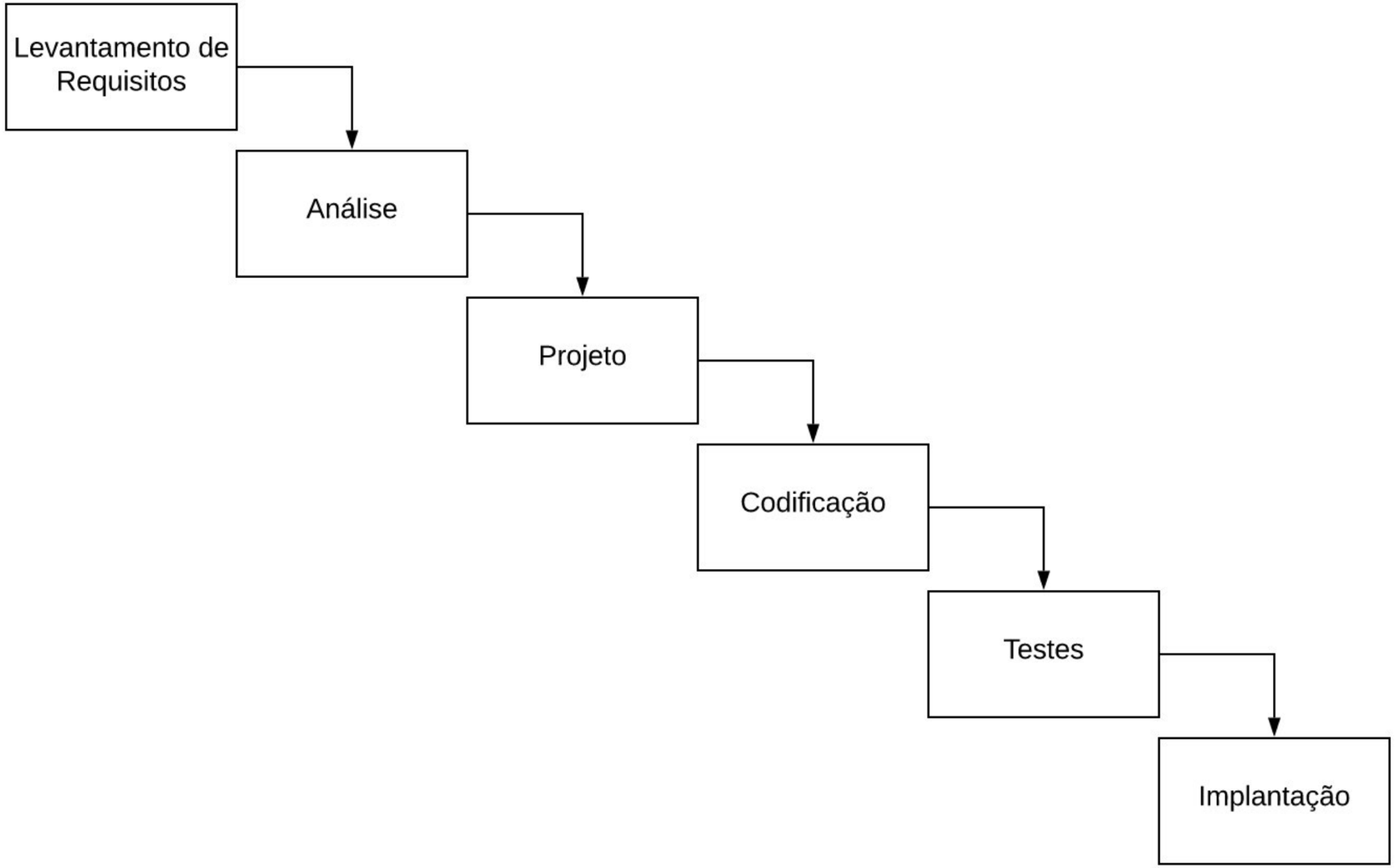
Modelo em Cascata

Inspirado em processos usados em engenharias tradicionais, como Civil, Mecânica, Elétrica, etc. Proposto na década de 70 e muito usado até ~1990. Adotado pelo Depto de Defesa Norte-Americano (1985).

AL

by Aimar Martins Lopes

Modelo em Cascata

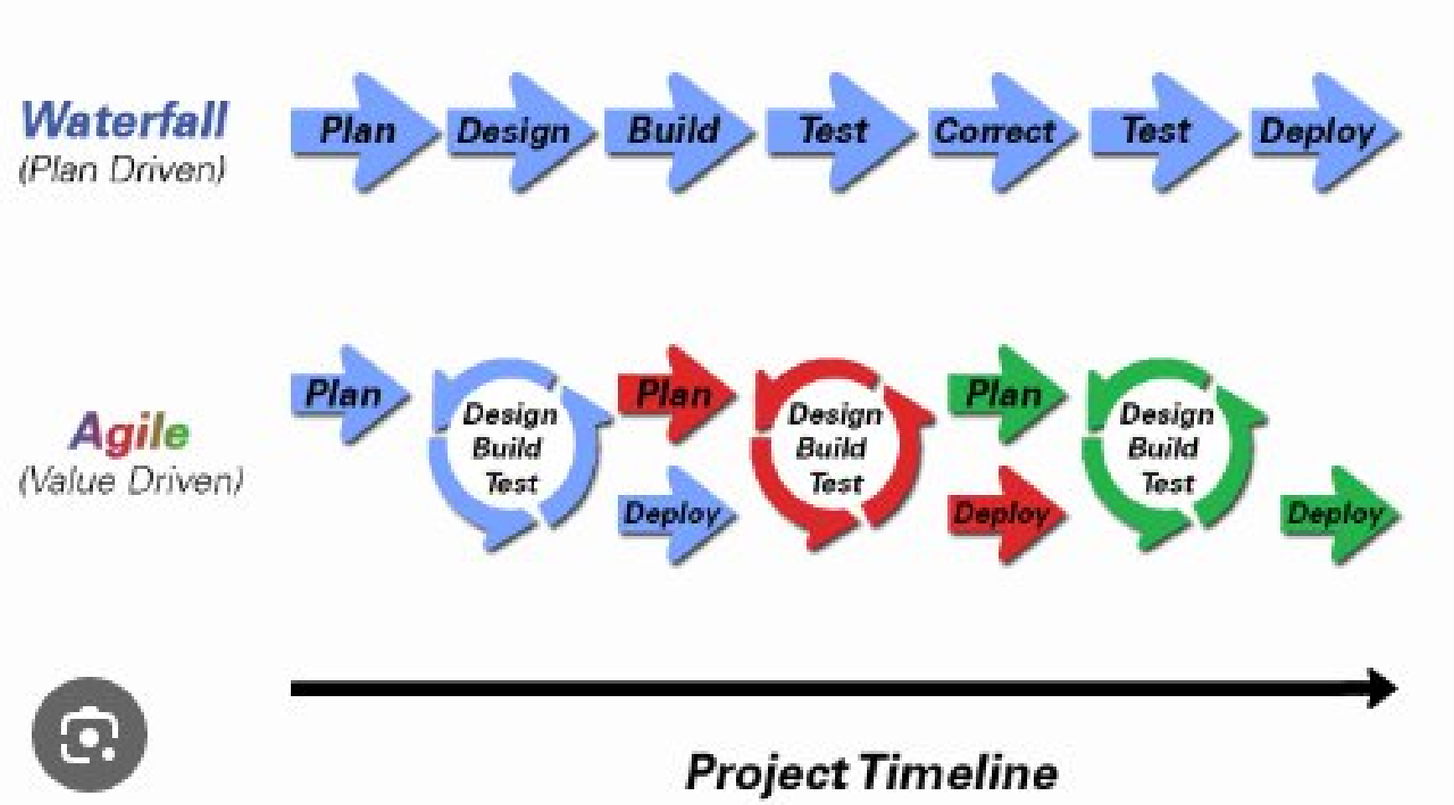


Modelo em Cascata

O modelo em cascata, também conhecido como modelo clássico ou modelo tradicional, é uma abordagem sequencial e linear para o desenvolvimento de software. Nesse modelo, o processo de desenvolvimento é dividido em etapas distintas, como especificação de requisitos, design, implementação, teste e manutenção. Cada etapa deve ser concluída antes de passar para a próxima, e o feedback de uma etapa não pode ser incorporado às etapas anteriores.

O modelo em cascata é caracterizado por uma abordagem rígida e inflexível, onde as mudanças de requisitos durante o desenvolvimento são difíceis de serem incorporadas. Isso pode levar a atrasos, custos adicionais e insatisfação do cliente, pois o software entregue pode não atender às suas necessidades atuais.

Modelo em Cascata
X
Modelo ágil



1. O **Modelo em Cascata** é uma abordagem tradicional de desenvolvimento de software, onde as fases do projeto são executadas sequencialmente.

2. Neste modelo, cada fase deve ser concluída antes que a próxima possa começar, criando uma **sequência linear** de atividades.

3. As principais fases do Modelo em Cascata são: **Análise de Requisitos, Projeto, Implementação, Testes e Manutenção**.

Problemas com Modelo Waterfall

Requisitos mudam com frequência

Os requisitos do software mudam constantemente, o que torna difícil para a equipe de desenvolvimento acompanhar essas mudanças no modelo Waterfall.

Levantamento completo de requisitos demanda tempo

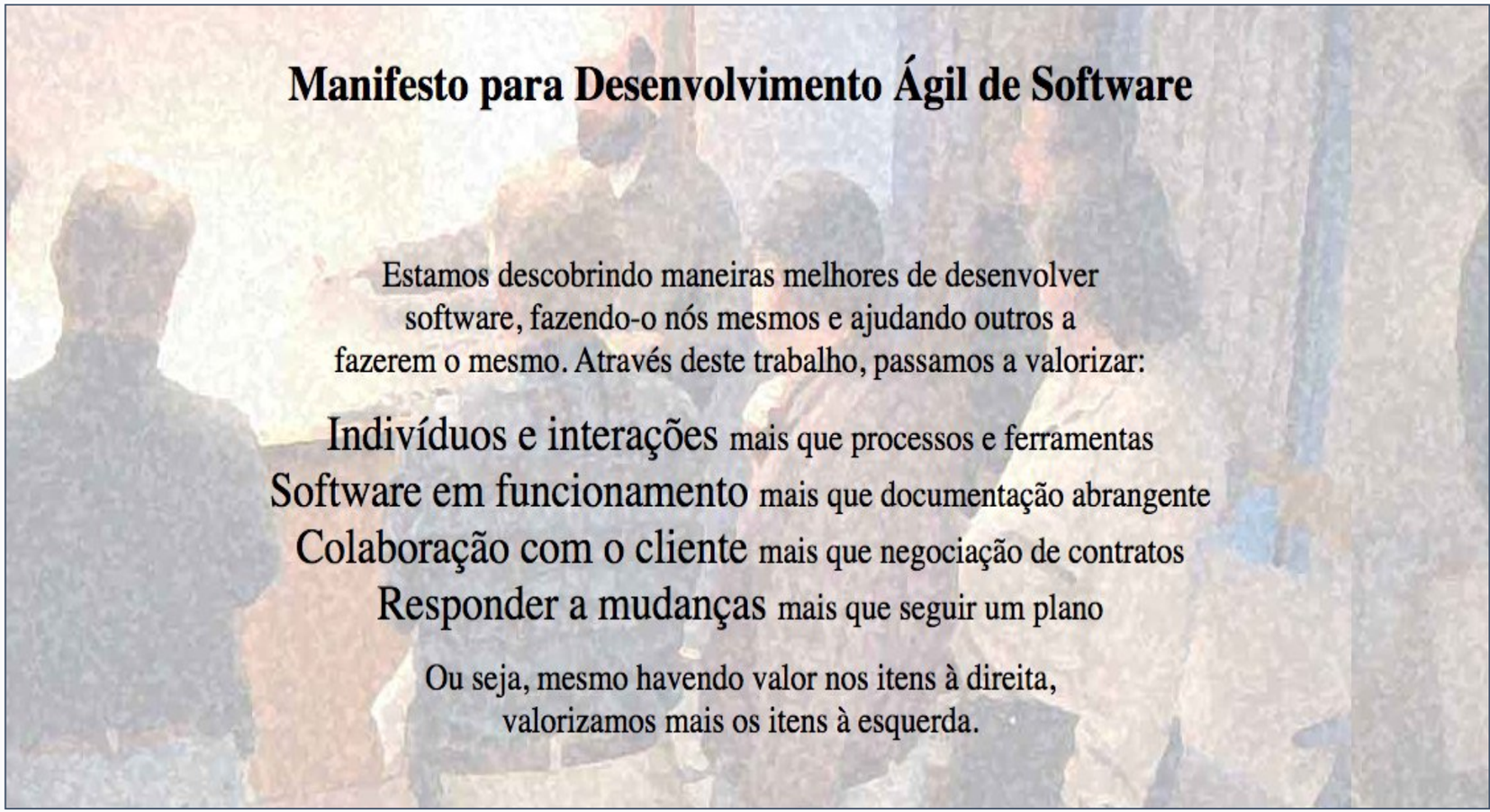
O levantamento completo de requisitos no início do projeto consome muito tempo, o que pode atrasar o início do desenvolvimento.

Quando ficar pronto, o "mundo já mudou"

Devido ao longo tempo de desenvolvimento, quando o software finalmente é entregue, o mundo e as necessidades do cliente já podem ter mudado.

Manifesto Ágil 2001

Um encontro de 17 engenheiros de software em Utah criticou os modelos sequenciais e pesados existentes. Eles propuseram um novo modelo, mais incremental e iterativo, que ficou conhecido como o Manifesto Ágil.



https://siamchamnankit.co.th/history-some-pictures-and-pdfs-of-the-agile-manifesto-meeting-on-2001-a33c40bcc2b

Made with Gamma

https://siamchamnankit.co.th/history-some-pictures-and-pdfs-of-the-agile-manifesto-meeting-on-2001-a33c40bcc2b

Manifesto Ágil (2001)

Valores do Manifesto Ágil

Em 2001, um grupo de desenvolvedores de software se reuniu para discutir novas abordagens para o desenvolvimento de software. Eles criaram o Manifesto Ágil, que estabelece quatro valores principais:

Indivíduos e Interações

Valorizar indivíduos e interações mais do que processos e ferramentas.

Software Funcional

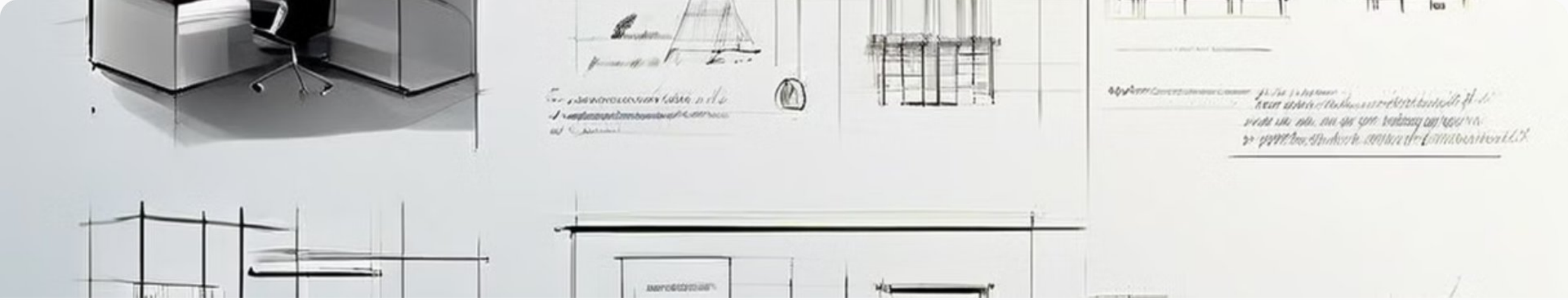
Valorizar software em funcionamento mais do que documentação abrangente.

Colaboração com o Cliente

Valorizar a colaboração com o cliente mais do que a negociação de contratos.

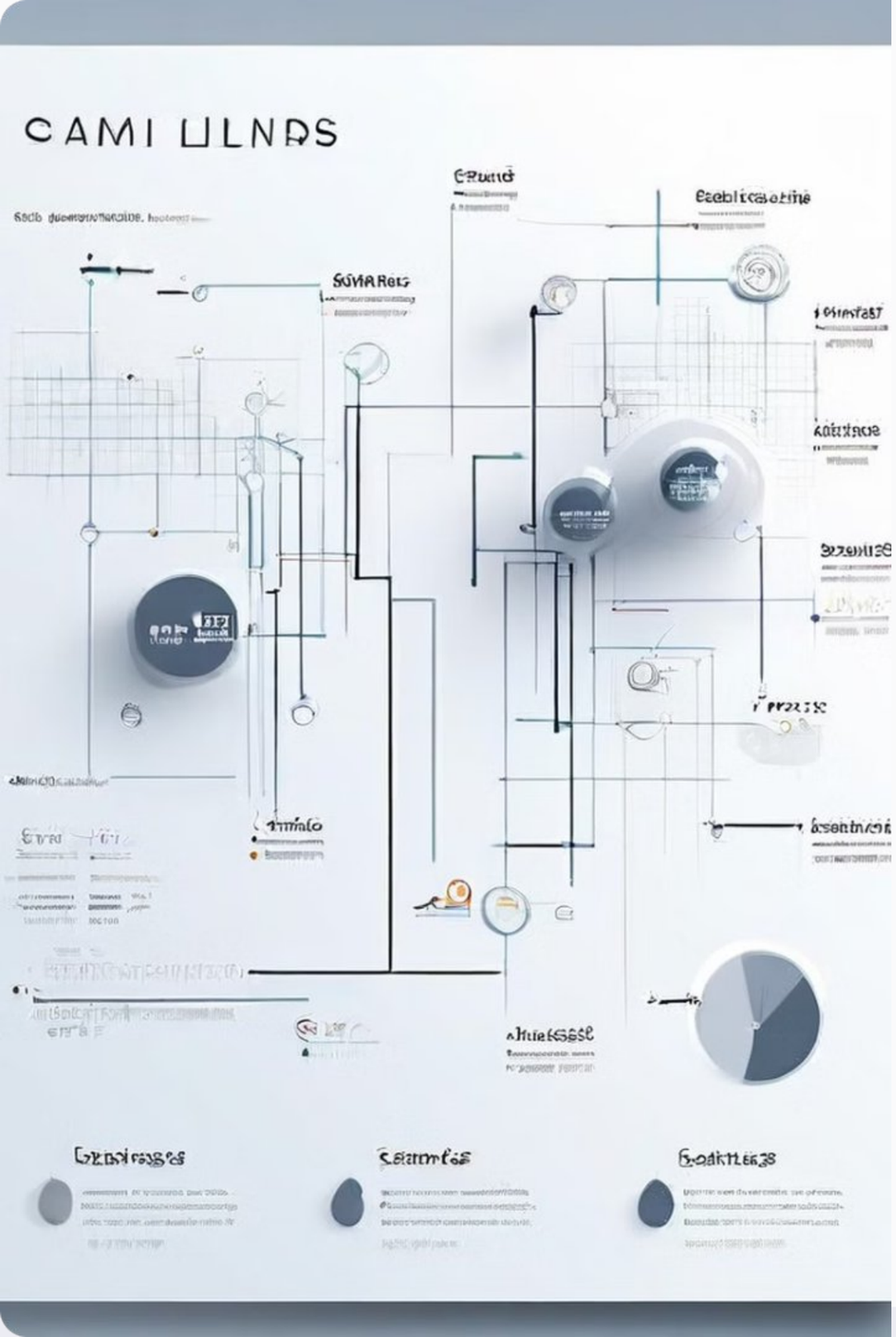
Resposta a Mudanças

Valorizar a resposta a mudanças mais do que seguir um plano.



Modelagem de Software

- **Modelo = representação mais alto nível do que o código**
- **Modelo pode ser usado para:**
- explicar o "design" para os devs(desenvolvedores)
- documentar o projeto
- facilitar compreensão e manutenção
- Modelos como sketches



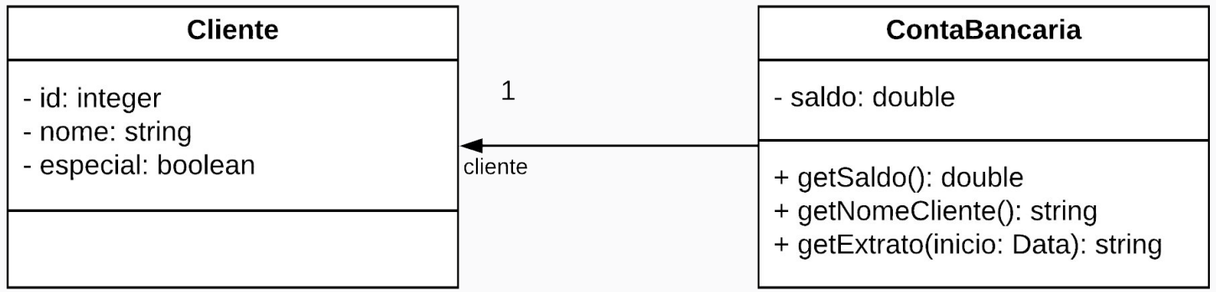
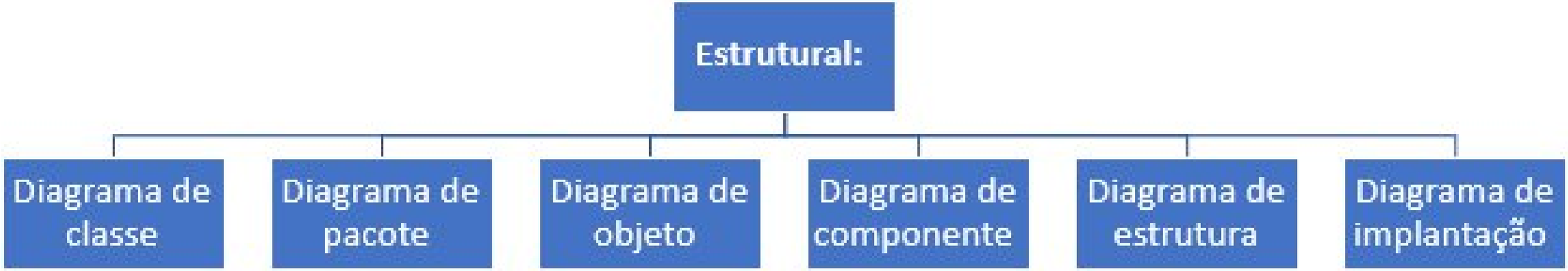


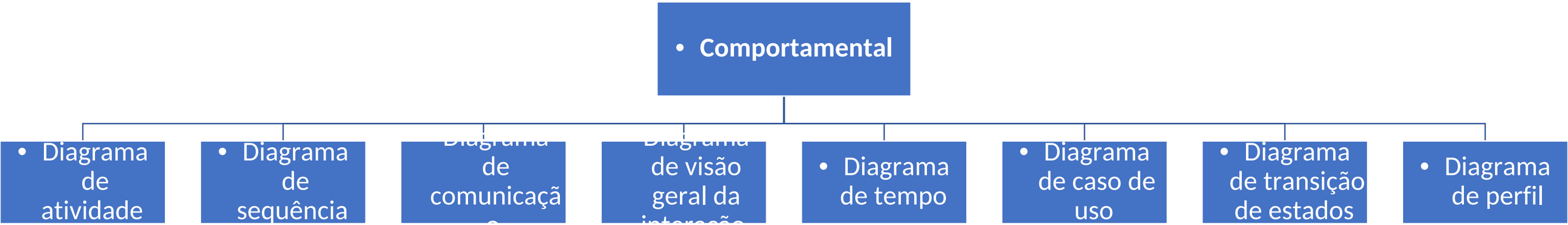
Diagrama de Classes UML

Unified Modeling Language (UML)

A Linguagem de Modelagem Unificada (UML) é uma linguagem gráfica para modelagem de software. O Diagrama de Classes UML é uma das principais ferramentas utilizadas para representar a estrutura de um sistema de software, mostrando as classes, seus atributos e métodos, bem como os relacionamentos entre elas.

Unified Modeling Language (UML)





Qualidade de Software

Qualidade Externa

Correção, robustez, extensibilidade, reusabilidade, eficiência, compatibilidade, **facilidade de uso**, portabilidade

Qualidade Interna

Modularidade, legibilidade, testabilidade, **manutenibilidade**, etc

Prática Profissional

Ética

Os profissionais de software devem seguir princípios éticos, como respeitar a privacidade dos usuários, evitar conflitos de interesse e agir com integridade.

Certificações

Obter certificações relevantes pode ajudar a demonstrar competência e conhecimento na área de desenvolvimento de software.

Regulamentação da Profissão

Em alguns países, a profissão de desenvolvedor de software pode ser regulamentada, com requisitos específicos para exercê-la.

Cursos de graduação

Cursos de graduação em Ciência da Computação, Engenharia de Software, ADS ou áreas afins podem fornecer uma base sólida para a prática profissional.

Currículo de Referência

Existem currículos de referência que orientam o desenvolvimento de programas de graduação na área.

Aspectos Econômicos

Explore os principais aspectos econômicos a serem considerados no desenvolvimento de software, incluindo retorno de investimento, modelos de monetização e precificação.

Para finalizar: Tipos ABC de sistemas

A classificação proposta por Bertrand Meyer divide os sistemas de software em três tipos principais:

Sistemas C (Casuais) e práticas deste curso

Tipo muito comum de sistema

Sistemas pequenos, sem muita importância. Podem ter bugs; às vezes, são descartáveis.

Desenvolvidos por 1-2 engenheiros

Não se beneficiam dos princípios. Risco: "over-engineering".

Exemplos de Sistemas Casuais

Jogos Casuais

Os jogos casuais são projetados para serem fáceis de aprender e jogar, com mecânicas simples e objetivos claros. Eles geralmente têm gráficos 2D coloridos e um estilo visual leve e descontraído, atraindo um público amplo e diversificado.

Aplicativos Casuais

Aplicativos casuais são aqueles projetados para uso rápido e ocasional, como jogos de quebra-cabeça, redes sociais e ferramentas de produtividade leve. Eles geralmente têm uma interface intuitiva e são fáceis de usar, permitindo que as pessoas se envolvam com eles de maneira descontraída e informal.

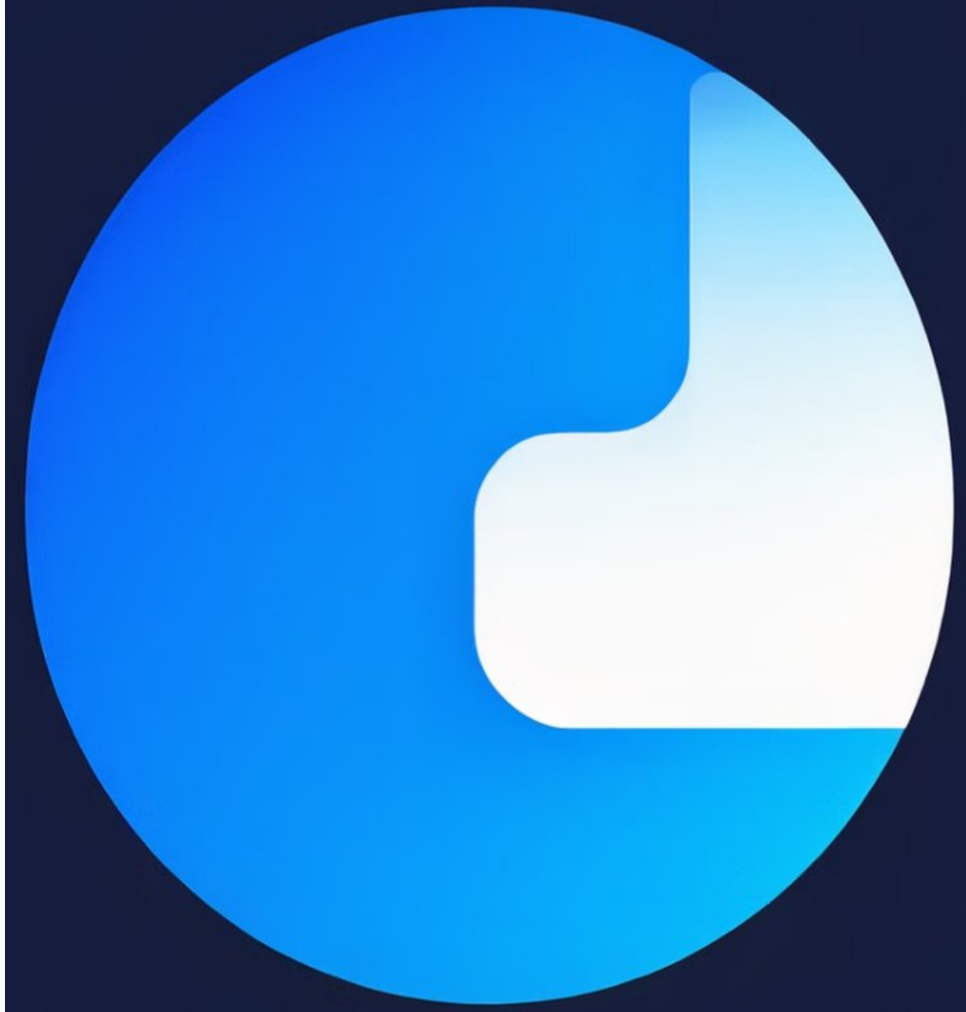
Atividades Casuais

Atividades casuais são aquelas que as pessoas realizam de forma descontraída e informal, como jogar jogos de tabuleiro com amigos, assistir a filmes leves ou participar de atividades sociais sem compromisso. Essas atividades são importantes para o bem-estar e a descompressão, proporcionando momentos de diversão e relaxamento.

Exemplos de Sistemas Casuais

https://github.com/mtoyoda/sl

O repositório https://github.com/mtoyoda/sl contém um programa chamado "sl" (Steam Locomotive), que é uma pequena animação de uma locomotiva a vapor que é exibida quando o usuário digita "sl" no terminal. Essa ferramenta foi criada por Toyoda Masashin, um desenvolvedor japonês, como uma brincadeira para distrair usuários que digitaram "ls" (list) por engano.



Sistemas B (Business)

Sistemas B (Business) são sistemas projetados para atender às necessidades de negócios, como gerenciamento de estoque, contabilidade, recursos humanos e planejamento estratégico. Esses sistemas são construídos para lidar com grandes volumes de dados, processos complexos e requisitos de conformidade.

Algumas características-chave dos sistemas B incluem:

- Escalabilidade para acomodar o crescimento do negócio
- Integração com outros sistemas empresariais
- Funcionalidades avançadas de relatórios e análise de dados
- Segurança e controles robustos para proteger informações confidenciais

Sistemas B (Business)

1

Sistemas importantes para uma organização

Os sistemas B (Business) são fundamentais para o funcionamento eficiente de uma organização. Eles desempenham um papel crucial no suporte às operações diárias e na tomada de decisões estratégicas.

2

Sistemas beneficiam-se do que veremos no curso

Esses sistemas podem se beneficiar enormemente das técnicas e práticas abordadas neste curso de Engenharia de Software. Isso pode ajudá-los a se tornarem ainda mais eficazes e alinhados com as necessidades da organização.

3

Risco: não usarem técnicas de ES e se tornarem um passivo, em vez de um ativo para as organizações

No entanto, se esses sistemas não adotarem as técnicas de Engenharia de Software adequadas, eles correm o risco de se tornarem um passivo, em vez de um ativo valioso para a organização. É essencial que esses sistemas sejam desenvolvidos e mantidos de acordo com as melhores práticas da Engenharia de Software.

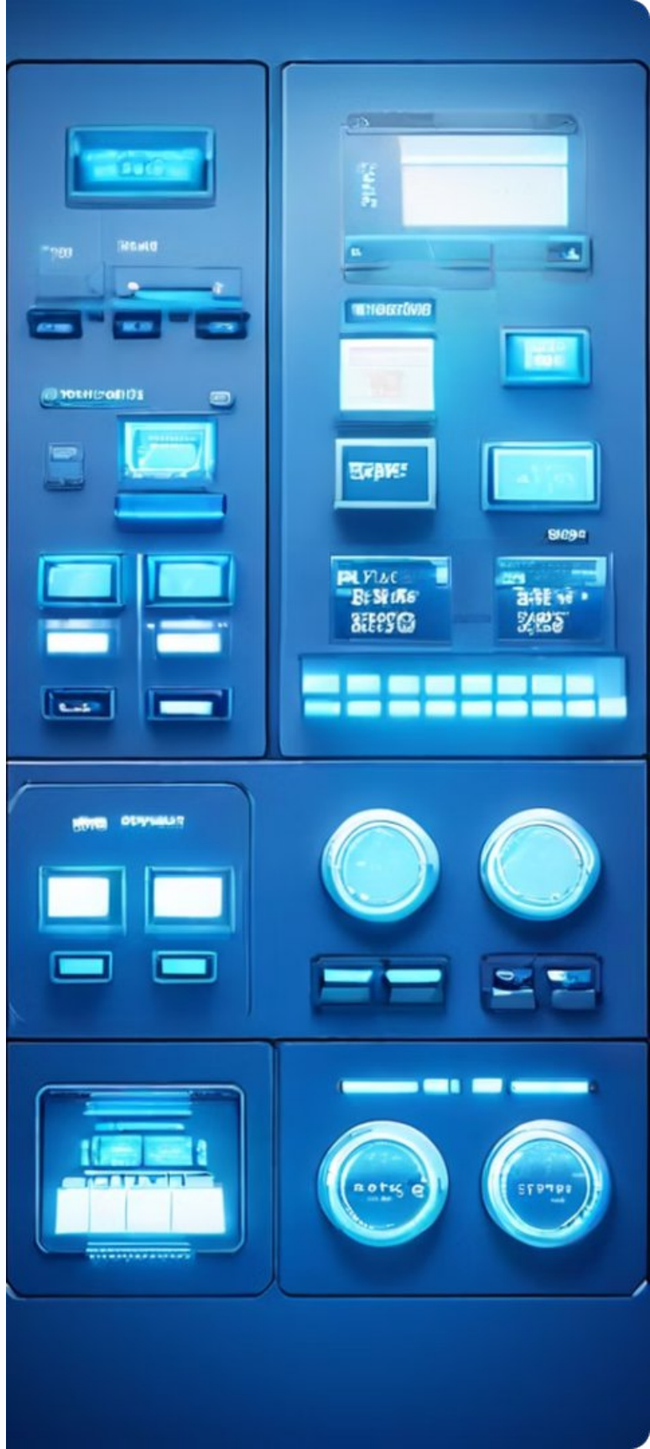
Sistemas A (Acute)

Sistemas de Missão Crítica

Sistemas onde nada pode dar errado, pois o custo é imenso, em termos de vidas humanas e/ou $$$. Também chamados sistemas de missão crítica.

Exemplos

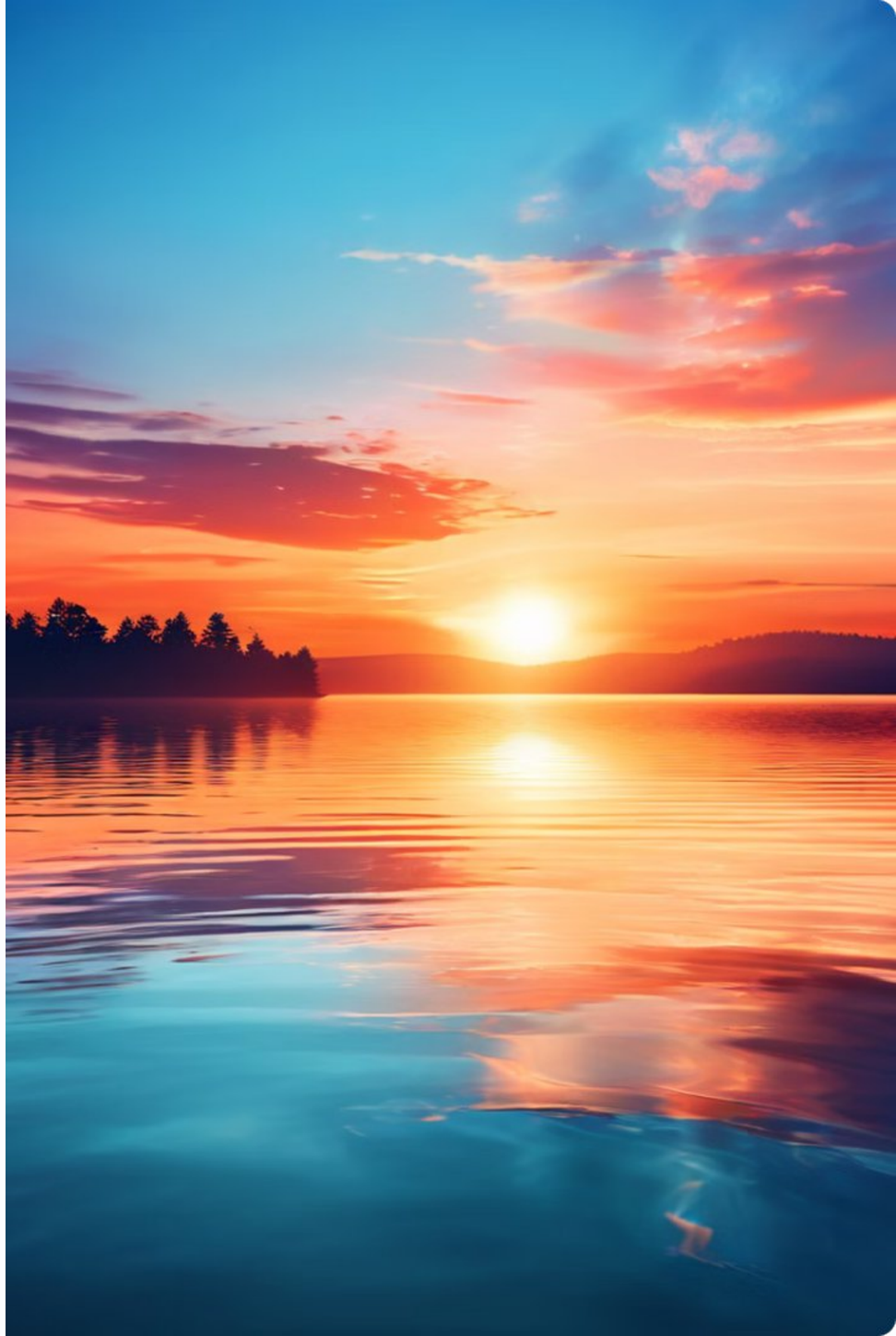
- Metrô
- Aviação
- Medicina



Sistemas A (Acute)

Requerem certificações

Fora do escopo do nosso curso

Fim

Obrigado!